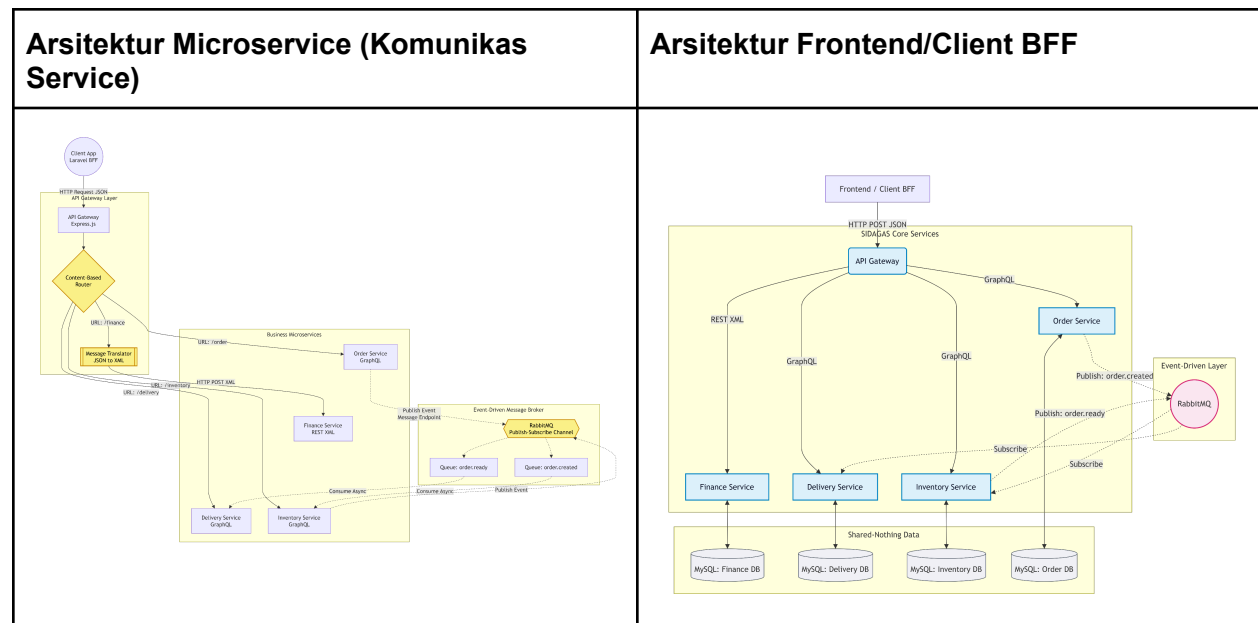


DOKUMENTASI SISTEM SIDAGAS (SISTEM INFORMASI DAGANG SYAHRIZAL GALON)

Untuk Memenuhi Tugas Besar Mata Kuliah Enterprise Application Integration

Disusun Oleh Kelompok 9 SI4804 :
Syahrizal Maulani 102022400226
Briantara Pradhipta 102022400041
Muhammad Farrel Sijji 102022400113
Aufa Rifqi Azzamy Syauqi 102022400079

ARSITEKTUR



ERD

orders (order_db)	
id	INT
customer_name	VARCHAR(255)
item_name	VARCHAR(255)
quantity	INT
status	VARCHAR(50)
created_at	TIMESTAMP
Indexes	

inventory (inventory_db)	
id	INT
item_name	VARCHAR(255)
stock	INT
updated_at	TIMESTAMP
Indexes	

deliveries (delivery_db)	
id	INT
order_id	INT
customer_name	VARCHAR(255)
address	VARCHAR(255)
status	VARCHAR(50)
created_at	TIMESTAMP
Indexes	

transactions (finance_db)	
id	INT
order_id	INT
amount	DECIMAL(10,2)
method	VARCHAR(50)
status	VARCHAR(50)
created_at	TIMESTAMP
Indexes	

Sistem ini menggunakan arsitektur database terdistribusi di mana setiap microservice mengelola databasenya sendiri (*Database-per-service*). Berikut rancangan tabel database relasional untuk masing-masing service:

1. Database: `order\_db` (Tabel: `orders`)

Menyimpan informasi transaksi pemesanan yang diajukan oleh pelanggan.

Skema Kolom:

- `id` (INT, Primary Key, Auto Increment)
- `customer\_name` (VARCHAR 255, Not Null)
- `item\_name` (VARCHAR 255, Not Null)
- `quantity` (INT, Not Null)
- `status` (VARCHAR 50, Default: 'pending')
- `created\_at` (TIMESTAMP, Default: Current Timestamp)

2. Database: `inventory\_db` (Tabel: `inventory`)

Menyimpan data stok barang dagang (Galon, Gas LPG).

Skema Kolom:

- `id` (INT, Primary Key, Auto Increment)

- `item\_name` (VARCHAR 255, Unique, Not Null)
- `stock` (INT, Default: 0, Not Null)
- `updated\_at` (TIMESTAMP, Default: Current/On Update Current Timestamp)

3. Database: `delivery\_db` (Tabel: `deliveries`)

Menyimpan jadwal dan status pengiriman barang oleh kurir/driver.

Skema Kolom:

- `id` (INT, Primary Key, Auto Increment)
- `order\_id` (INT, Not Null)
- `customer\_name` (VARCHAR 255, Not Null)
- `address` (VARCHAR 255, Default: 'Alamat belum diset')
- `status` (VARCHAR 50, Default: 'scheduled')
- `created\_at` (TIMESTAMP, Default: Current Timestamp)

4. Database: `finance\_db` (Tabel: `transactions`)

Menyimpan verifikasi transaksi pembayaran masuk (Data Legacy XML).

Skema Kolom:

- `id` (INT, Primary Key, Auto Increment)
- `order\_id` (INT, Not Null)
- `amount` (DECIMAL 10,2, Not Null)
- `method` (VARCHAR 50, Not Null)
- `status` (VARCHAR 50, Default: 'verified')
- `created\_at` (TIMESTAMP, Default: Current Timestamp)

Schema GraphQL



CHANGE SCHEMA

Type List

Search Schema...

Delivery

No Description

Item

No Description

Order

No Description

Query

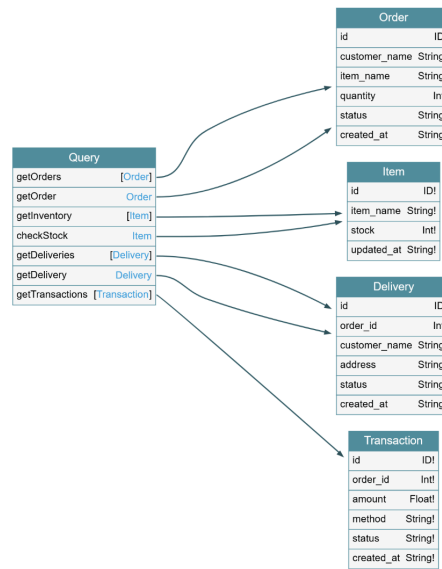
root

No Description

Transaction

No Description

✳ Powered by GraphQL Voyager



Query

☐ Sort by Alphabet ☒ Skip Relay ☒ Skip deprecated ☒ Show leaf fields





CHANGE SCHEMA

Type List

Search Schema...

Delivery

No Description

Item

No Description

Mutation

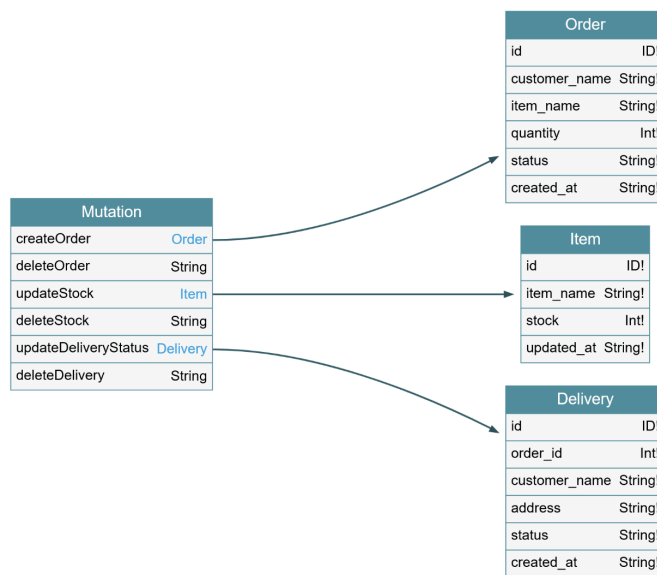
root

No Description

Order

No Description

✳ Powered by GraphQL Voyager



Mutation

☐ Sort by Alphabet ☒ Skip Relay ☒ Skip deprecated ☒ Show leaf fields



SCREENSHOOT MUTATION

GraphQL

Prettify

Merge

Copy

History

< Docs

```
1 mutation {
2   createOrder(customer_name: "Budi Santoso", item_name: "Galon Aqua", quantity: 5) {
3     id
4     customer_name
5     item_name
6     quantity
7     status
8     created_at
9   }
10 }
```

```
{
  "data": {
    "createOrder": {
      "id": "1",
      "customer_name": "Budi Santoso",
      "item_name": "Galon Aqua",
      "quantity": 5,
      "status": "pending",
      "created_at": "2026-06-18T12:47:22.963Z"
    }
  }
}
```

QUERY VARIABLES

SCREENSHOOT QUERY

GraphQL

Prettify

Merge

Copy

History

< Docs

```
1 query {
2   getOrders {
3     id
4     customer_name
5     item_name
6     quantity
7     status
8   }
9 }
```

```
{
  "data": {
    "getOrders": [
      {
        "id": "1",
        "customer_name": "Budi Santoso",
        "item_name": "Galon Aqua",
        "quantity": 5,
        "status": "pending"
      }
    ]
  }
}
```

QUERY VARIABLES

SCREENSHOT CLIENT

Dashboard Utama(Katalog & Riwayat)

-Katalog Produk



KatalogLacak Pesanan



Logout

Katalog Produk

Pesan kebutuhan air minum dan gas Anda sekarang, kami antar sampai depan pintu.



Galon Aqua

Rp 20.000

1

+ Keranjang



Gas LPG 3Kg

Rp 18.000

1

+ Keranjang

© 2026 SIDAGAS Microservices. Tugas EAL.

-Lacak Pesanan/Riwayat



KatalogLacak Pesanan



Logout

Lacak Pesanan

Pantau status pesanan Anda secara real-time.

Muat Ulang

ORD-2Invalid DateMenunggu Diproses



Gas LPG 3Kg

1 x Barang

DIBUAT

SIAP KIRIM

DI JALAN

SELESAI

ORD-1Invalid DateMenunggu Diproses



Galon Aqua

1 x Barang

DIBUAT

SIAP KIRIM

DI JALAN

SELESAI

Form Pemesanan

KatalogLacak Pesanan

Logout

Checkout Pesanan

Selesaikan pesanan Anda dengan memilih metode pengiriman dan pembayaran.

Detail Pengiriman

Metode Penerimaan

Diantar ke Rumah (Delivery)

Nama Penerima

Pelanggan Setia

Alamat Lengkap

telkom

Metode Pembayaran

Tunai (COD)
Bayar di tempat

Transfer Bank
BCA, Mandiri, dll

QRIS
OVO, Gopay, Dana

Ringkasan Belanja

Gas LPG 3Kg 1 x Rp 18.000	Rp 18.000
Subtotal	Rp 18.000
Ongkos Kirim	Gratis
Total	Rp 18.000

Bayar & Buat Pesanan →

Notifikasi Pesanan Berhasil

KatalogLacak Pesanan

Logout

Checkout Pesanan

Selesaikan pesanan Anda dengan memilih metode pengiriman dan pembayaran.

Detail Pengiriman

Metode Penerimaan

Diantar ke Rumah (Delivery)

Alamat Lengkap

telkom

Metode Pembayaran

Tunai (COD)
Bayar di tempat

Transfer Bank
BCA, Mandiri, dll

QRIS
OVO, Gopay, Dana

Ringkasan Belanja

Gas LPG 3Kg Rp 18.000	Rp 18.000
Subtotal	Rp 18.000
Ongkos Kirim	Gratis
Total	Rp 18.000

Memproses...



Pesanan Berhasil!

Terima kasih, pesanan Anda sedang kami proses.

OK

Panduan Menjalankan Proyek (Langkah Rinci)

Pastikan aplikasi Docker Desktop (atau Docker Engine) telah terinstal dan dalam status Running di komputer Anda.

Tahap 1: Persiapan Repository

Clone Repository (Jika belum) Buka terminal/CMD Anda, lalu jalankan:

```
git clone https://github.com/[username_anda]/SIDAGAS.git  
cd SIDAGAS/EAI_UAS
```

Konfigurasi Environment Salin file konfigurasi lingkungan. Secara default, sistem sudah bisa jalan langsung tanpa pengubahan. (Jika file .env belum ada, buatlah berdasarkan .env.example).

Tahap 2: Menyalakan Infrastruktur (Docker Compose)

Dari dalam direktori EAI_UAS (yang memuat file docker-compose.yml), ketikkan satu baris sakti berikut di terminal Anda:

```
docker compose up --build -d
```

Penjelasan Perintah:

up: Memerintahkan Docker menyalakan seluruh sistem.

--build: Memaksa Docker untuk membaca ulang kode Javascript Node.js Anda dan membangun image baru jika ada perubahan.

-d: (Detached Mode) Menjalankan server di latar belakang agar terminal Anda tidak terkunci dan tetap bisa digunakan.

Tahap 3: Verifikasi Sistem Telah Aktif

Membangun puluhan komponen dan 4 database MySQL secara bersamaan membutuhkan waktu sekitar 30 - 60 detik pada saat pertama kali berjalan (tergantung kecepatan laptop).

Cara mengecek apakah semua sudah Running:

Buka Docker Desktop -> Tab Containers. Pastikan grup eai_uas memiliki 10 kontainer dengan status ikon Hijau (Running).

Cek Koneksi Gateway: Buka Web Browser, ketik <http://localhost:3000>. Jika terbuka informasi server (atau pesan respons JSON), artinya sistem online.

Cek RabbitMQ: Buka <http://localhost:15672>. Login dengan Username guest dan Password guest. Jika muncul dashboard statistik pesan, sistem Broker sehat.

Tahap 4: Mengintegrasikan ke Frontend (Laravel UI)

Setelah Backend Microservices Docker Anda menyala sehat:

Buka tab terminal baru.

Masuk ke folder Laravel:

```
cd ../Frontend
```

Instal library PHP dan NPM:

```
composer install
```

```
npm install
```


Jalankan Laravel:

`php artisan serve`

Lalu di terminal baru, jalankan CSS Compiler:

`npm run dev`

Sukses! Buka browser ke <http://localhost:8000/login>. Seluruh transaksi yang Anda lakukan di antarmuka Laravel sekarang secara otomatis dialirkan, diubah formatnya, dan disalurkan ke kontainer-kontainer Microservice Docker secara real-time.